INSTITUTE
OF LAW,
POLITICS AND
DEVELOPMENT



Sant'Anna
School of Advanced Studies – Pisa

Quantitative Methods

Module I, Session 1: Causal Identification and Inference

Prof. Kevin Koehler
kevin.koehler@santannapisa.it

Section 1

1. Why quantitative methods?

1.1 What we actually want to know

Democracies repress their citizens less than autocracies (Poe and Tate 1994). As a description, this is a correlation. But it begs a causal question: *if a country became more democratic, would it repress less?*

- Does joining the International Criminal Court change how a state deals with the United States?

We are rarely content with associations. The question is "**what would have happened otherwise?**"

1.1 What we actually want to know

Democracies repress their citizens less than autocracies (Poe and Tate 1994). As a description, this is a correlation. But it begs a causal question: *if a country became more democratic, would it repress less?*

- Does joining the International Criminal Court change how a state deals with the United States?
- Does a favourable view of the US among a country's public make its government more likely to side with Washington?

We are rarely content with associations. The question is "**what would have happened otherwise?**"

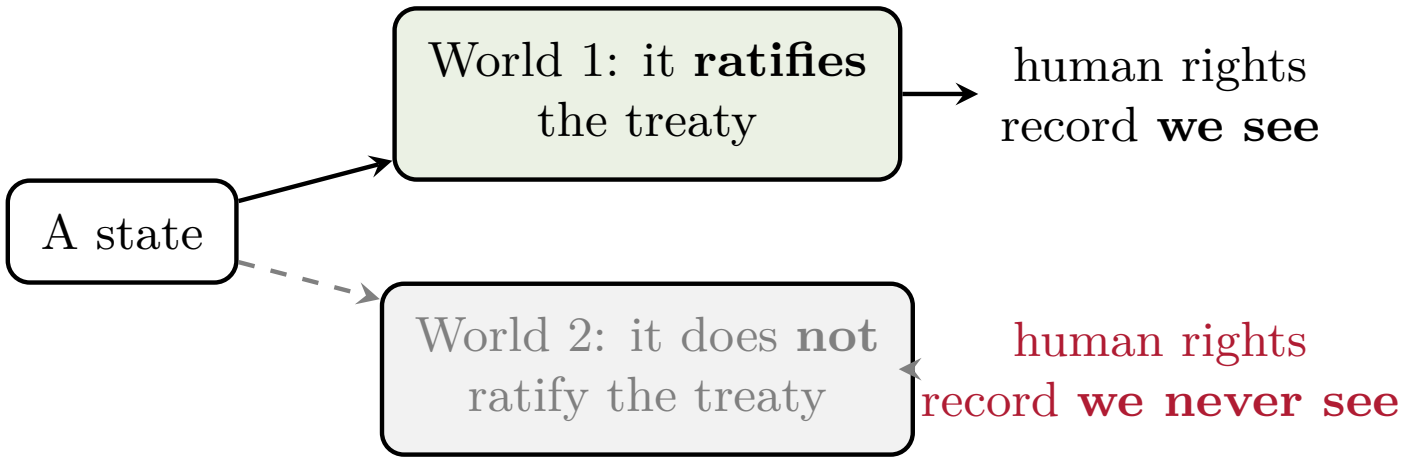
1.1 What we actually want to know

Democracies repress their citizens less than autocracies (Poe and Tate 1994). As a description, this is a correlation. But it begs a causal question: *if a country became more democratic, would it repress less?*

- Does joining the International Criminal Court change how a state deals with the United States?
- Does a favourable view of the US among a country's public make its government more likely to side with Washington?
- Does ratifying a human rights treaty improve a state's human rights practices?

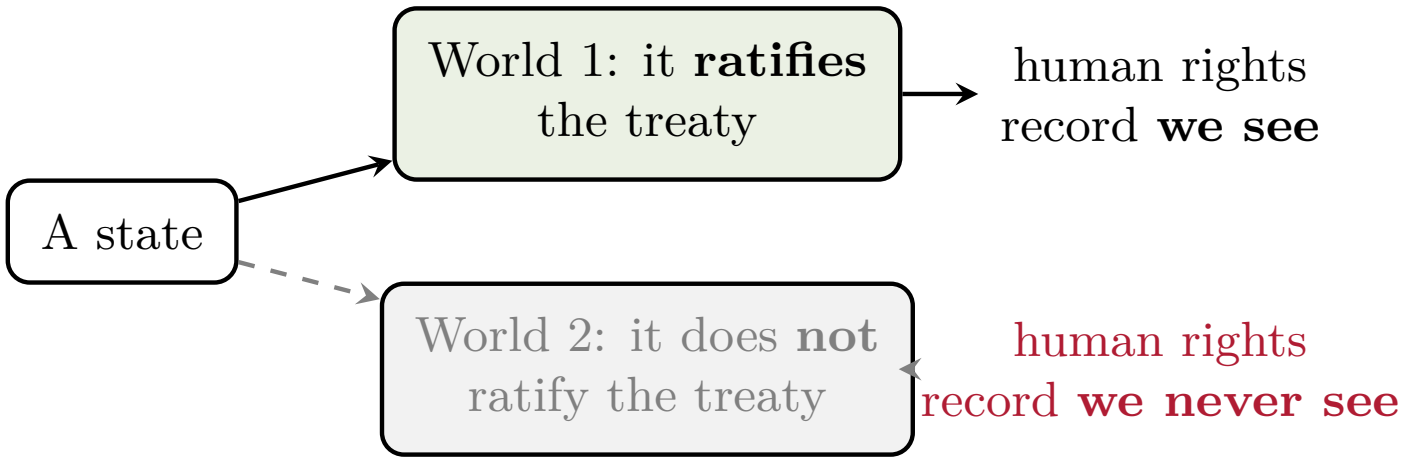
We are rarely content with associations. The question is "**what would have happened otherwise?**"

1.2 Two versions of one world



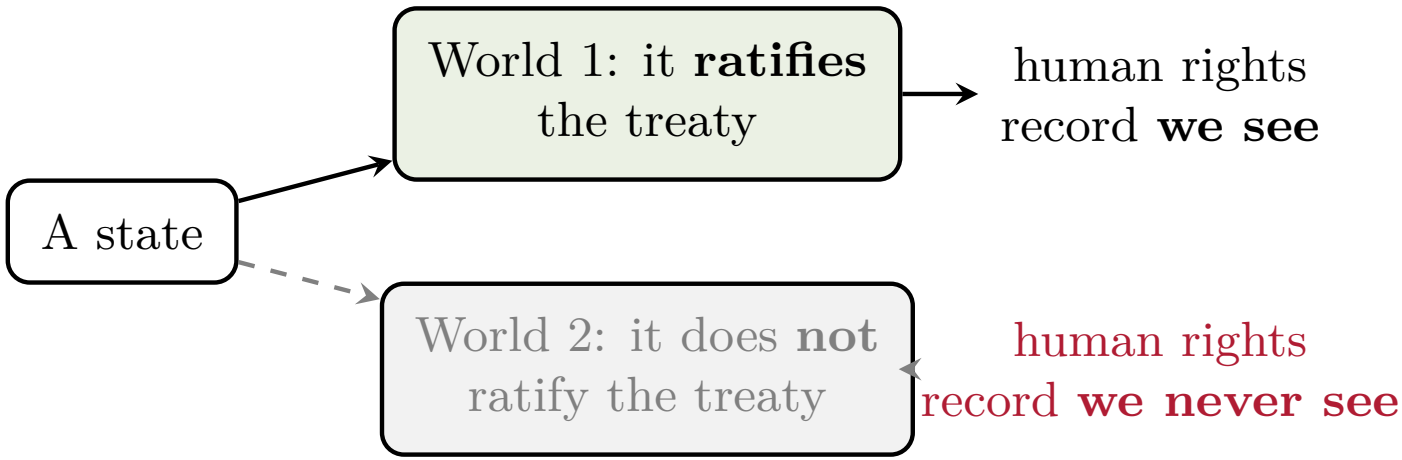
- The **effect** of ratifying is the difference between the two records: the *same* state, at the *same* moment.

1.2 Two versions of one world



- The **effect** of ratifying is the difference between the two records: the *same* state, at the *same* moment.
- **The fundamental problem of causal inference:** only one world ever happens (Holland 1986). The other is a **counterfactual**.

1.2 Two versions of one world



- The **effect** of ratifying is the difference between the two records: the *same* state, at the *same* moment.
- **The fundamental problem of causal inference:** only one world ever happens (Holland 1986). The other is a **counterfactual**.
- So we aim for an **average effect** across many units — and decide *whose* average before touching any data.

1.3 What the data can and cannot tell us

Picture every state in two rows (did / did not ratify) and two columns (the record *with* the treaty, the record *without* it):

	With treaty	**Without treaty**
States that ratified	**observed**	**?**
States that did not	**?**	**observed**

- The average effect needs all four cells. The data fill two.

1.3 What the data can and cannot tell us

Picture every state in two rows (did / did not ratify) and two columns (the record *with* the treaty, the record *without* it):

	With treaty	**Without treaty**
States that ratified	**observed**	**?**
States that did not	**?**	**observed**

- The average effect needs all four cells. The data fill two.
- The two question marks are what the ratifiers would have looked like *without* the treaty, and the non-ratifiers *with* it.

1.3 What the data can and cannot tell us

Picture every state in two rows (did / did not ratify) and two columns (the record *with* the treaty, the record *without* it):

	With treaty	**Without treaty**
States that ratified	**observed**	**?**
States that did not	**?**	**observed**

- The average effect needs all four cells. The data fill two.
- The two question marks are what the ratifiers would have looked like *without* the treaty, and the non-ratifiers *with* it.
- More data does not help: every new state lands in an observed cell. The gap can only be closed by **assumptions** (Keele 2015).

1.4 Identification

An effect is **identified** if, with unlimited data, our comparison would give us the right answer (Keele 2015).

- The key assumption: whether a state gets the treatment has nothing to do with how it would have fared either way — *as if a coin had been flipped*. Plus: no spillovers, one state's ratification does not change another's record.

1.4 Identification

An effect is **identified** if, with unlimited data, our comparison would give us the right answer (Keele 2015).

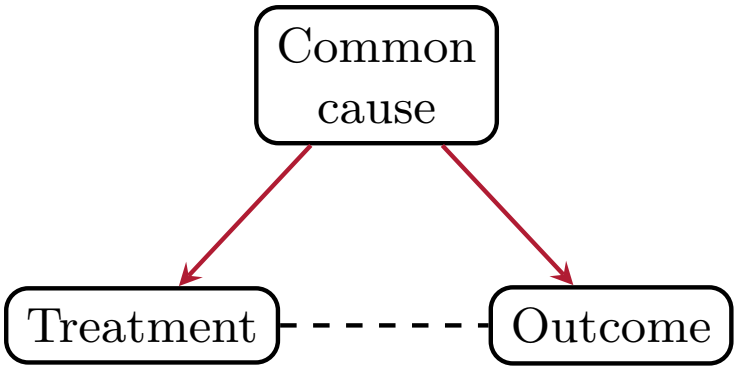
- The key assumption: whether a state gets the treatment has nothing to do with how it would have fared either way — *as if a coin had been flipped*. Plus: no spillovers, one state's ratification does not change another's record.
- If that holds, the non-ratifiers are a fair stand-in for the ratifiers' missing world, and vice versa. The question marks can be filled in, and the effect is simply **the difference between the two groups' averages**.

1.4 Identification

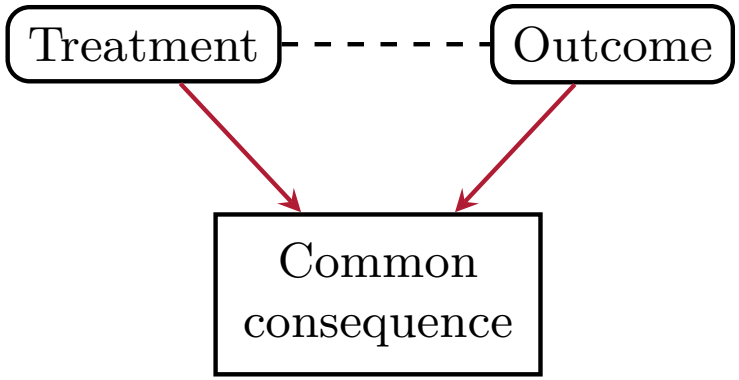
An effect is **identified** if, with unlimited data, our comparison would give us the right answer (Keele 2015).

- The key assumption: whether a state gets the treatment has nothing to do with how it would have fared either way — *as if a coin had been flipped*. Plus: no spillovers, one state's ratification does not change another's record.
- If that holds, the non-ratifiers are a fair stand-in for the ratifiers' missing world, and vice versa. The question marks can be filled in, and the effect is simply **the difference between the two groups' averages**.
- Identification is a property of the **research design** — not of the model or the software. Most of these assumptions cannot be tested with the data.

1.5 Two threats to identification



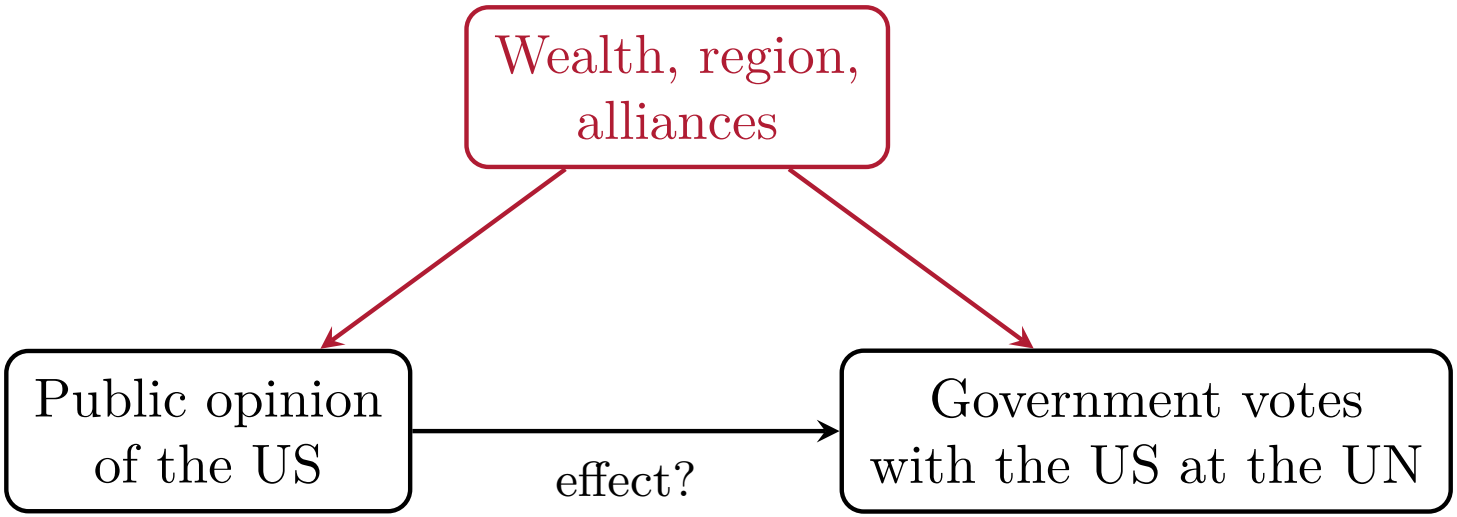
(a) **Confounding**: something drives both



(b) **Selection**: we filter or adjust on a result of both

In both pictures, treatment and outcome move together (dashed line) even if the treatment does nothing. Confounding is the familiar problem. Selection is subtler: controlling for something the treatment itself changed, or studying only cases picked because of their outcome, can *create* a spurious link where there was none (Keele 2015).

1.6 An example: soft power



Rich European NATO members tend both to like the US and to vote with it. The raw correlation between opinion and voting mixes any real effect with that common cause. This is the question Goldsmith & Horiuchi (2012) ask — and their data are the data we use for the first half of the course.

1.7 Ways of making the comparison fair

Strategy	**What you have to believe**	**In this course**
The researcher flips the coin *(experiment)*	Nothing more: chance decided who was treated	the benchmark (Session 7)
The world flips the coin *(natural experiment)*	Something outside anyone's control decided, as good as by chance	beyond this course
Measure the common causes *("selection on observables")*	We have measured and adjusted for everything that drives both	Sessions 3–4, 7–8
Compare units with themselves over time *(panel designs)*	What we cannot measure does not change over time, or groups would have moved in parallel	Sessions 5–6, 8

Regression, matching and weighting are ways of **computing** a comparison, not of making it fair. None can rescue a weak design (Keele 2015).

1.8 What causal inference does not do

- Causal inference answers closed questions: *does* X affect Y, and by how much? The answer is "yes", "no", or a number.

Quantitative methods are **one** way of establishing causal effects. They work alongside theory, qualitative evidence — often decisive for defending a research design — and other forms of explanation.

1.8 What causal inference does not do

- Causal inference answers closed questions: *does* X affect Y, and by how much? The answer is "yes", "no", or a number.
- That is not yet an **explanation**. To explain, the effect has to be tied to a generalization and a mechanism — that is, to theory (Dowding and Miller 2026).

Quantitative methods are **one** way of establishing causal effects. They work alongside theory, qualitative evidence — often decisive for defending a research design — and other forms of explanation.

1.8 What causal inference does not do

- Causal inference answers closed questions: *does* X affect Y, and by how much? The answer is "yes", "no", or a number.
- That is not yet an **explanation**. To explain, the effect has to be tied to a generalization and a mechanism — that is, to theory (Dowding and Miller 2026).
- Some good explanations are not causal at all: **constitutive** (what a democracy is, how a network is organised) and **by constraint** (equilibria, results from social choice).

Quantitative methods are **one** way of establishing causal effects. They work alongside theory, qualitative evidence — often decisive for defending a research design — and other forms of explanation.

1.9 Two problems, not one

Identification

Would our comparison give the right answer with unlimited data?

→ research design
→ Modules II–IV

Inference

How close to the right answer are we with the data we actually have?

→ probability
→ Module I, starting today

No statistical technique can save a poor research design. The most credible causal findings need the least statistics (Keele 2015). But even a perfect design gives us an **estimate** from a limited sample — and to tell an effect from noise we need to know how estimates bounce around. That is what **distributions** describe.

1.10 Four rules for the term

1. **Say what you want to know before choosing how to compute it.** The average effect for whom?

These rules also govern what you accept from an AI agent. It will cheerfully report "the effect" of every variable in a model. Deciding what the output means is your job.

1.10 Four rules for the term

1. **Say what you want to know before choosing how to compute it.** The average effect for whom?
2. **Say why your comparison is fair, and defend it** — with substantive, often qualitative, knowledge of the case.

These rules also govern what you accept from an AI agent. It will cheerfully report "the effect" of every variable in a model. Deciding what the output means is your job.

1.10 Four rules for the term

1. **Say what you want to know before choosing how to compute it.** The average effect for whom?
2. **Say why your comparison is fair, and defend it** — with substantive, often qualitative, knowledge of the case.
3. **Use causal language only for effects your design identifies.** In a regression that is usually one number; the numbers next to the control variables mean nothing causally (Keele, Stevenson and Elwert 2020).

These rules also govern what you accept from an AI agent. It will cheerfully report "the effect" of every variable in a model. Deciding what the output means is your job.

1.10 Four rules for the term

1. **Say what you want to know before choosing how to compute it.** The average effect for whom?
2. **Say why your comparison is fair, and defend it** — with substantive, often qualitative, knowledge of the case.
3. **Use causal language only for effects your design identifies.** In a regression that is usually one number; the numbers next to the control variables mean nothing causally (Keele, Stevenson and Elwert 2020).
4. **Then estimate, and say how uncertain you are.**

These rules also govern what you accept from an AI agent. It will cheerfully report "the effect" of every variable in a model. Deciding what the output means is your job.

1.11 The inferential problem

We almost never observe what we want to know.

- We want to know something about a **population**: all states, all judgments of a court, all voters.

Probability is the bridge. It tells us how estimates behave *if* we knew the population — and that is exactly what we need to reason backwards from an estimate to the population.

1.11 The inferential problem

We almost never observe what we want to know.

- We want to know something about a **population**: all states, all judgments of a court, all voters.
- We observe a **sample**: 43 countries in a survey, the treaties that happen to be coded, the respondents who answered.

Probability is the bridge. It tells us how estimates behave *if* we knew the population — and that is exactly what we need to reason backwards from an estimate to the population.

1.11 The inferential problem

We almost never observe what we want to know.

- We want to know something about a **population**: all states, all judgments of a court, all voters.
- We observe a **sample**: 43 countries in a survey, the treaties that happen to be coded, the respondents who answered.
- The true value in the population is fixed but unknown. What we compute from the sample is an **estimate**: known, but it would come out differently in another sample.

Probability is the bridge. It tells us how estimates behave *if* we knew the population — and that is exactly what we need to reason backwards from an estimate to the population.

1.12 Two kinds of numbers

	The true value	**Our estimate**
Average UN agreement with the US	across all states	across the states in our data
Share signing an Article 98 deal	among all states	among those we coded
Effect of opinion on UN voting	the real effect	the number in our regression
Status	unknown, fixed	known, varies from sample to sample

Confidence intervals, p-values and standard errors are all statements about the **right-hand column** that we use to make claims about the **middle** one.

Section 2

2. The module

2.1 Quantitative Methods: where we are

Module I: Fundamentals of Statistical Reasoning

1. **Session 1 (today):** why we do this, and how estimates vary

Identification is the aim; inference is the foundation. Everything that follows in the course — regression, panel data, causal inference — rests on the two ideas in this module: that our data are a **sample**, and that samples behave in **predictable** ways.

2.1 Quantitative Methods: where we are

Module I: Fundamentals of Statistical Reasoning

1. **Session 1 (today):** why we do this, and how estimates vary
2. **Session 2:** hypothesis testing and confidence intervals

Identification is the aim; inference is the foundation. Everything that follows in the course — regression, panel data, causal inference — rests on the two ideas in this module: that our data are a **sample**, and that samples behave in **predictable** ways.

2.2 What this module is *not*

- It is not a mathematics course. We care about the *logic*, not the proofs. Today's slides contain no formulas.

It does assume you will check things. With an agent doing the typing, the skill is knowing what right looks like.

2.2 What this module is *not*

- It is not a mathematics course. We care about the *logic*, not the proofs. Today's slides contain no formulas.
- It is not a coding course. An AI agent will write most of the code — but you will read every line of it.

It does assume you will check things. With an agent doing the typing, the skill is knowing what right looks like.

2.2 What this module is *not*

- It is not a mathematics course. We care about the *logic*, not the proofs. Today's slides contain no formulas.
- It is not a coding course. An AI agent will write most of the code — but you will read every line of it.
- It does not assume prior statistical training. If you have none, work through Dean & Illowsky (2018) alongside the sessions.

It does assume you will check things. With an agent doing the typing, the skill is knowing what right looks like.

2.3 How we code in this course

An **AI coding agent** (OpenCode) writes the code. You are responsible for knowing what it does and whether it is right.

- Syntax is no longer the bottleneck: an agent writes working R code faster than you can look it up.

The working rule: you must be able to explain every line you submit and justify every analytical choice in it. "The AI did it" is not an answer to "why did you drop these observations?"

2.3 How we code in this course

An **AI coding agent** (OpenCode) writes the code. You are responsible for knowing what it does and whether it is right.

- Syntax is no longer the bottleneck: an agent writes working R code faster than you can look it up.
- What has not changed: whether missing cases were silently dropped, whether a function got the wrong kind of input, whether a number supports a causal sentence. The code runs either way.

The working rule: you must be able to explain every line you submit and justify every analytical choice in it. "The AI did it" is not an answer to "why did you drop these observations?"

2.3 How we code in this course

An **AI coding agent** (OpenCode) writes the code. You are responsible for knowing what it does and whether it is right.

- Syntax is no longer the bottleneck: an agent writes working R code faster than you can look it up.
- What has not changed: whether missing cases were silently dropped, whether a function got the wrong kind of input, whether a number supports a causal sentence. The code runs either way.
- Every lab has the same rhythm: **specify** → **let the agent work** → **check against something independent**. In this module, what we know about distributions is that independent check.

The working rule: you must be able to explain every line you submit and justify every analytical choice in it. "The AI did it" is not an answer to "why did you drop these observations?"

Section 3

3. Lab: getting set up

3.1 Three tools

R is the **language**. It is what actually computes.

RStudio is the **interface**: editor, console, data and plots in one window.

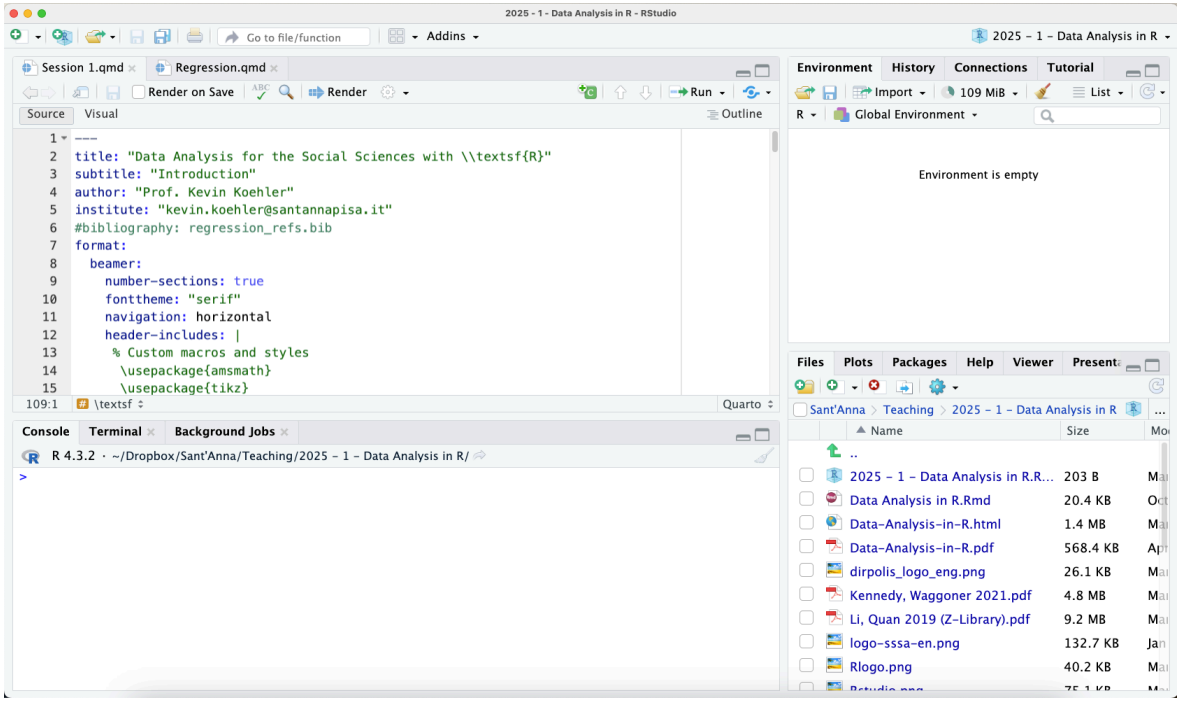
OpenCode is the **agent**. Unlike a chat window, it reads your files, writes new ones and runs commands on your machine.



1. R: `cran.r-project.org`
2. RStudio Desktop: `posit.co/downloads`
3. OpenCode: `opencode.ai/download`

Install R first, then RStudio.

3.2 The RStudio window



Next to the Console is a **Terminal** tab. The Console talks to R; the Terminal talks to your computer. OpenCode can run in the Terminal and runs R through it (or use the OpenCode desktop app, pointed at the same folder).

3.3 Your first task: a project

Projects keep everything for one piece of work in one place, and tell R where to look for files. Use them from day one — it is the single cheapest thing you can do for reproducibility.

1. Create a folder on your computer for this course, e.g. `PhD-Quant`

```
getwd()
```

3.3 Your first task: a project

Projects keep everything for one piece of work in one place, and tell R where to look for files. Use them from day one — it is the single cheapest thing you can do for reproducibility.

1. Create a folder on your computer for this course, e.g. `PhD-Quant`
2. In RStudio: `File > New Project > Existing Directory`

```
getwd()
```

3.3 Your first task: a project

Projects keep everything for one piece of work in one place, and tell R where to look for files. Use them from day one — it is the single cheapest thing you can do for reproducibility.

1. Create a folder on your computer for this course, e.g. `PhD-Quant`
2. In RStudio: `File > New Project > Existing Directory`
3. Navigate to the folder and create the project

```
getwd()
```

3.3 Your first task: a project

Projects keep everything for one piece of work in one place, and tell R where to look for files. Use them from day one — it is the single cheapest thing you can do for reproducibility.

1. Create a folder on your computer for this course, e.g. `PhD-Quant`
2. In RStudio: `File > New Project > Existing Directory`
3. Navigate to the folder and create the project
4. Confirm where you are:

```
getwd()
```

3.4 Starting the agent

1. Open the **Terminal** tab in RStudio and type `opencode`. The free models work straight away; to use Claude, GPT or Gemini instead, type `/connect`.

`AGENTS.md` holds standing instructions the agent reads at the start of every session: make random draws repeatable, report how many rows every step keeps, never drop missing values silently. `log.md` is yours: one line per session on something the agent got wrong and how you caught it.

3.4 Starting the agent

1. Open the **Terminal** tab in RStudio and type `opencode`. The free models work straight away; to use Claude, GPT or Gemini instead, type `/connect`.
2. Test it: *"List the files in this project and say what each one is for."*

`AGENTS.md` holds standing instructions the agent reads at the start of every session: make random draws repeatable, report how many rows every step keeps, never drop missing values silently. `log.md` is yours: one line per session on something the agent got wrong and how you caught it.

3.4 Starting the agent

1. Open the **Terminal** tab in RStudio and type `opencode`. The free models work straight away; to use Claude, GPT or Gemini instead, type `/connect`.
2. Test it: *"List the files in this project and say what each one is for."*
3. Check it can reach R: ask it to run `Rscript -e "sessionInfo()"`.

`AGENTS.md` holds standing instructions the agent reads at the start of every session: make random draws repeatable, report how many rows every step keeps, never drop missing values silently. `log.md` is yours: one line per session on something the agent got wrong and how you caught it.

3.4 Starting the agent

1. Open the **Terminal** tab in RStudio and type `opencode`. The free models work straight away; to use Claude, GPT or Gemini instead, type `/connect`.
2. Test it: *"List the files in this project and say what each one is for."*
3. Check it can reach R: ask it to run `Rscript -e "sessionInfo()"`.
4. Copy the course `AGENTS.md` into your project folder, and create an empty `log.md`.

`AGENTS.md` holds standing instructions the agent reads at the start of every session: make random draws repeatable, report how many rows every step keeps, never drop missing values silently. `log.md` is yours: one line per session on something the agent got wrong and how you caught it.

3.5 Staying in control

- **Plan before build.** Press `Shift+Tab` (terminal version; `Tab` in the desktop app) to switch the agent into *Plan* mode: it reads and proposes, but changes nothing. Switch back to *Build* once you agree with the plan.

3.5 Staying in control

- **Plan before build.** Press `Shift+Tab` (terminal version; `Tab` in the desktop app) to switch the agent into *Plan* mode: it reads and proposes, but changes nothing. Switch back to *Build* once you agree with the plan.
- **Read a script before you let it run.** Nothing runs that nobody has looked at.

3.5 Staying in control

- **Plan before build.** Press `Shift+Tab` (terminal version; `Tab` in the desktop app) to switch the agent into *Plan* mode: it reads and proposes, but changes nothing. Switch back to *Build* once you agree with the plan.
- **Read a script before you let it run.** Nothing runs that nobody has looked at.
- **The agent's account of its work is itself output to check.** Fluent explanations can be wrong: "removed missing values" may have removed nothing — or half your sample.

3.6 Reading R: what you need to recognise

You will rarely write R from scratch. You will read a lot of it.

```
ratifications <- c(12, 8, 31, 0, 5)  # store numbers
mean(ratifications)                  # apply a function
```

```
[1] 11.2
```

```
d <- data.frame(state = c("A", "B", "C"),
                ratified = c(1, 0, 1))
d$state[d$ratified == 1]   # filter a column
```

```
[1] "A" "C"
```

`<-` stores; `==` asks. `?mean` opens the help page — use it to check what the agent's functions do, rather than asking the agent.

3.7 Missing values

R is deliberately noisy about missing data, which is a feature:

```
x <- c(4, 7, NA, 2)
mean(x)
```

```
[1] NA
```

```
mean(x, na.rm = TRUE)
```

```
[1] 4.333333
```

Agents add `na.rm = TRUE` to make code run. Every time you see it, ask which cases just disappeared, and why. Which countries drop out of an analysis is a substantive question, not a technical one — we will come back to this in Session 2.

3.8 Packages

Functions live in **packages**. Your installation comes with base R; everything else you add from **CRAN**.

```
install.packages("tidyverse")    # once, ever
library(tidyverse)               # every session
```

For this course, ask the agent to install these — then check yourself that `library()` loads each one:

```
install.packages(c("tidyverse", "haven", "broom"))
```

`tidyverse` for data work and graphics, `haven` to read Stata and SPSS files (most replication data comes in those formats), `broom` to turn model output into tidy tables.

3.9 Lab checkpoint

Before we move on, everyone should have:

1. R and RStudio installed and opening without errors

Two minutes: compare with your neighbour. Fixing an installation now costs less than fixing it in week four.

3.9 Lab checkpoint

Before we move on, everyone should have:

1. R and RStudio installed and opening without errors
2. A project folder for the course, with `getwd()` pointing at it

Two minutes: compare with your neighbour. Fixing an installation now costs less than fixing it in week four.

3.9 Lab checkpoint

Before we move on, everyone should have:

1. R and RStudio installed and opening without errors
2. A project folder for the course, with `getwd()` pointing at it
3. OpenCode running, having listed your project files

Two minutes: compare with your neighbour. Fixing an installation now costs less than fixing it in week four.

3.9 Lab checkpoint

Before we move on, everyone should have:

1. R and RStudio installed and opening without errors
2. A project folder for the course, with `getwd()` pointing at it
3. OpenCode running, having listed your project files
4. `AGENTS.md` and an empty `log.md` in the project

Two minutes: compare with your neighbour. Fixing an installation now costs less than fixing it in week four.

3.9 Lab checkpoint

Before we move on, everyone should have:

1. R and RStudio installed and opening without errors
2. A project folder for the course, with `getwd()` pointing at it
3. OpenCode running, having listed your project files
4. `AGENTS.md` and an empty `log.md` in the project
5. `tidyverse`, `haven` and `broom` installed — and loaded by you

Two minutes: compare with your neighbour. Fixing an installation now costs less than fixing it in week four.

Section 4

4. Distributions: how often things happen

4.1 Things we cannot predict exactly

Much of what we study has an outcome we do not know in advance — but we know something about *how often* each outcome occurs.

- **Counts and yes/no outcomes:** the number of states signing a treaty in a year; the number of dissenting opinions in a judgment; a single vote.

The difference matters. For a count we can ask "how likely is *exactly* 3?". For a measurement we can only sensibly ask "how likely is it to fall *between* 3 and 4?" — the chance of hitting 3.000000… exactly is zero.

4.1 Things we cannot predict exactly

Much of what we study has an outcome we do not know in advance — but we know something about *how often* each outcome occurs.

- **Counts and yes/no outcomes:** the number of states signing a treaty in a year; the number of dissenting opinions in a judgment; a single vote.
- **Measurements:** GDP per capita; the share of a population holding an opinion; the length of a proceeding.

The difference matters. For a count we can ask "how likely is *exactly* 3?". For a measurement we can only sensibly ask "how likely is it to fall *between* 3 and 4?" — the chance of hitting 3.000000… exactly is zero.

4.2 Distributions

A **distribution** tells you which values can occur and how often.

- For **counts**, it is a bar chart: the height of each bar is the probability of that value.

Two numbers usually summarise a distribution: where its **centre** is (the average we would expect) and how **spread out** it is (the standard deviation: roughly, the typical distance from the centre).

4.2 Distributions

A **distribution** tells you which values can occur and how often.

- For **counts**, it is a bar chart: the height of each bar is the probability of that value.
- For **measurements**, it is a smooth curve: the *area* under the curve between two points is the probability of landing between them.

Two numbers usually summarise a distribution: where its **centre** is (the average we would expect) and how **spread out** it is (the standard deviation: roughly, the typical distance from the centre).

4.2 Distributions

A **distribution** tells you which values can occur and how often.

- For **counts**, it is a bar chart: the height of each bar is the probability of that value.
- For **measurements**, it is a smooth curve: the *area* under the curve between two points is the probability of landing between them.
- Either way, we can also ask for a running total: the chance of getting a value *at or below* some point.

Two numbers usually summarise a distribution: where its **centre** is (the average we would expect) and how **spread out** it is (the standard deviation: roughly, the typical distance from the centre).

4.3 Counting yes/no outcomes: the binomial

Flip a coin many times and count the heads. The **binomial** distribution tells you how likely each count is, given the number of flips and the chance of heads on each.

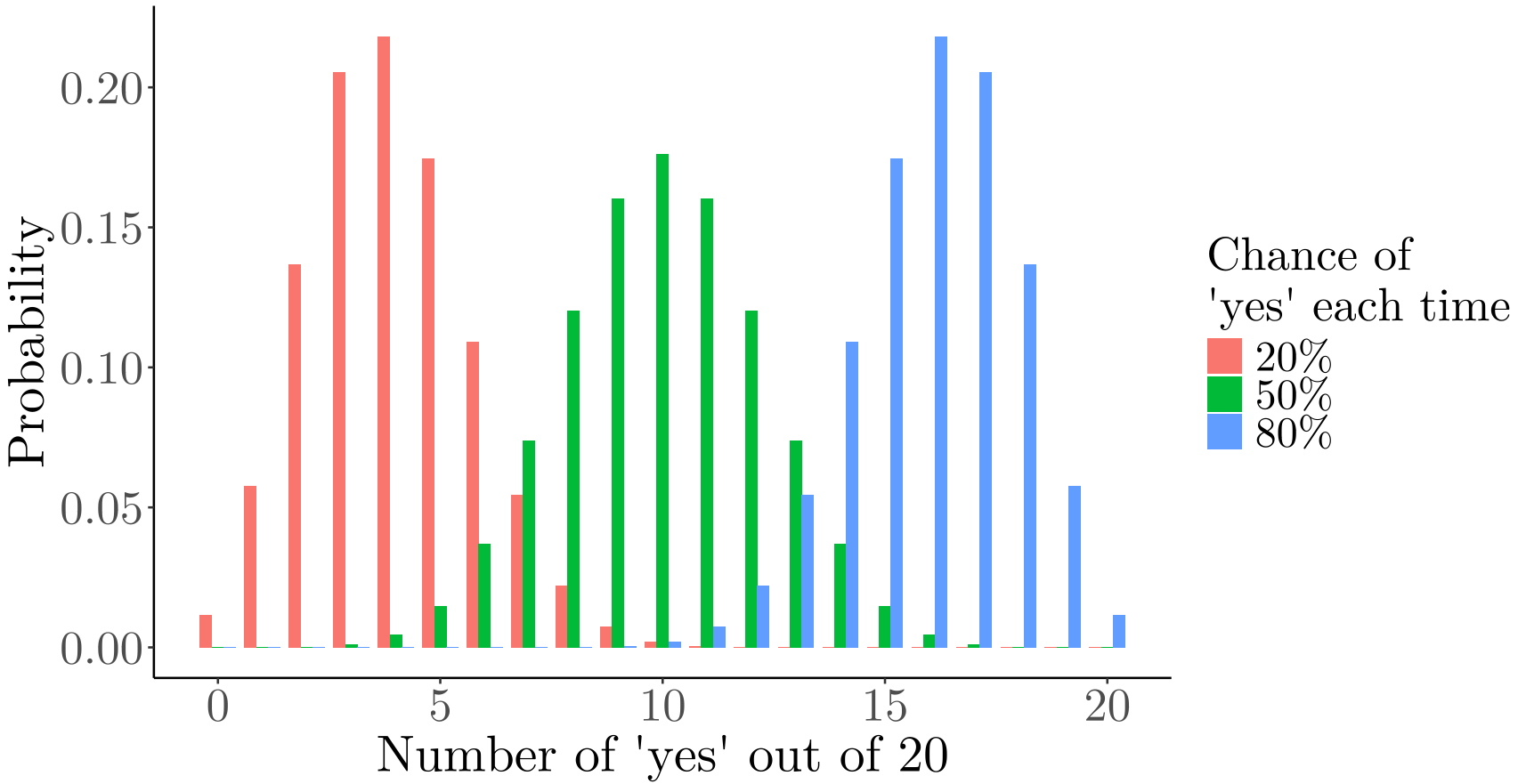
Example. Of the 216 states in the Goldsmith & Horiuchi data, 60 had signed an Article 98 agreement with the United States by 2003 — an agreement not to hand US nationals to the International Criminal Court. If each state had simply flipped a fair coin, how likely is a count as low as 60?

```
sum(dbinom(0:60, size = 216, prob = 0.5))
```

```
[1] 2.456826e-11
```

Practically impossible. Whatever drove these decisions, it was not a fair coin.

4.4 The binomial: what it looks like



4.5 The normal distribution: the bell curve

- Symmetric, with a single peak. Two numbers describe it completely: its **centre** and its **spread**.

It matters less because real variables follow a bell curve — most do not — and more because, as we will see in a moment, *averages from samples* do.

4.5 The normal distribution: the bell curve

- Symmetric, with a single peak. Two numbers describe it completely: its **centre** and its **spread**.
- About **two thirds** of values fall within one standard deviation of the centre; about **95%** within two.

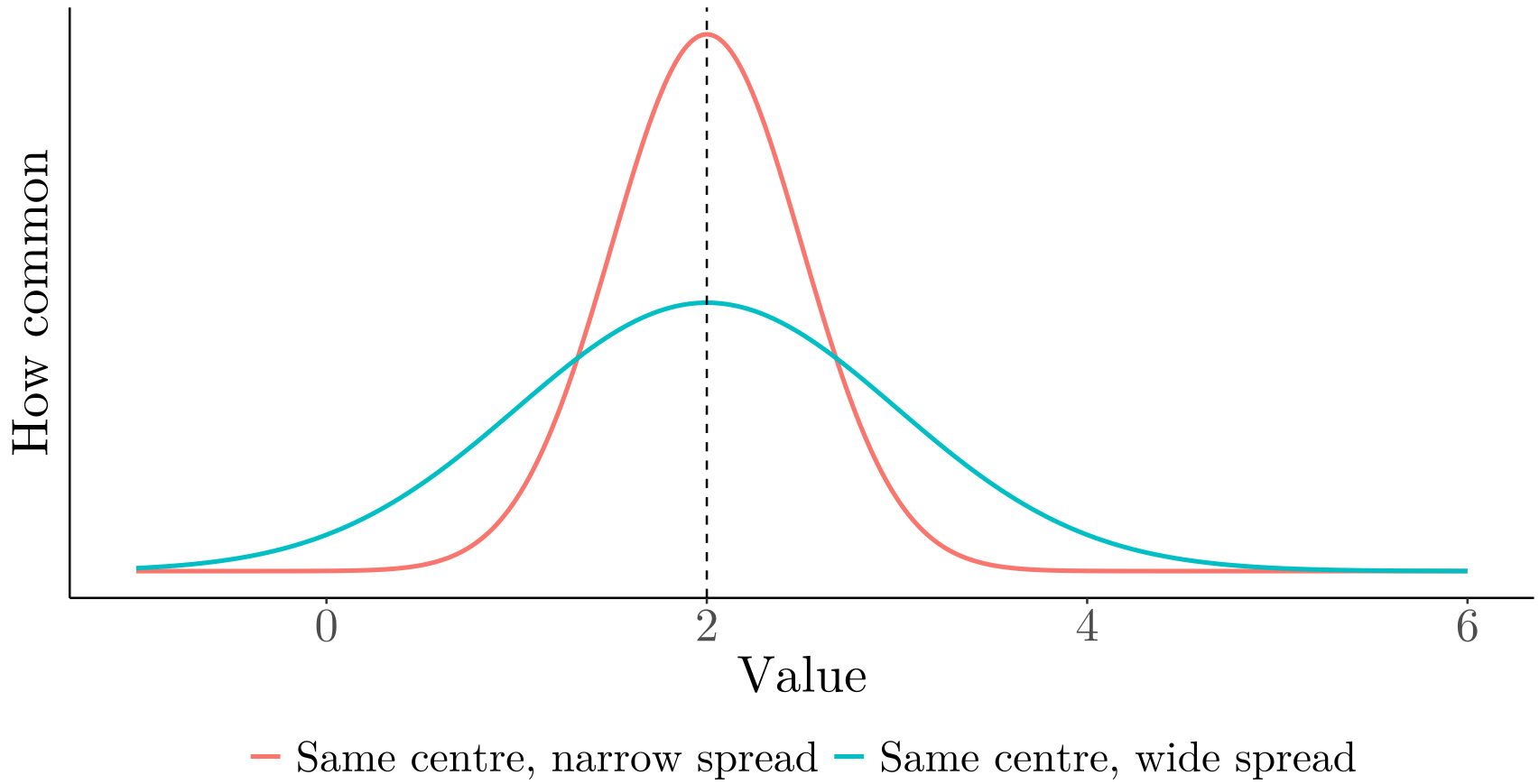
It matters less because real variables follow a bell curve — most do not — and more because, as we will see in a moment, *averages from samples* do.

4.5 The normal distribution: the bell curve

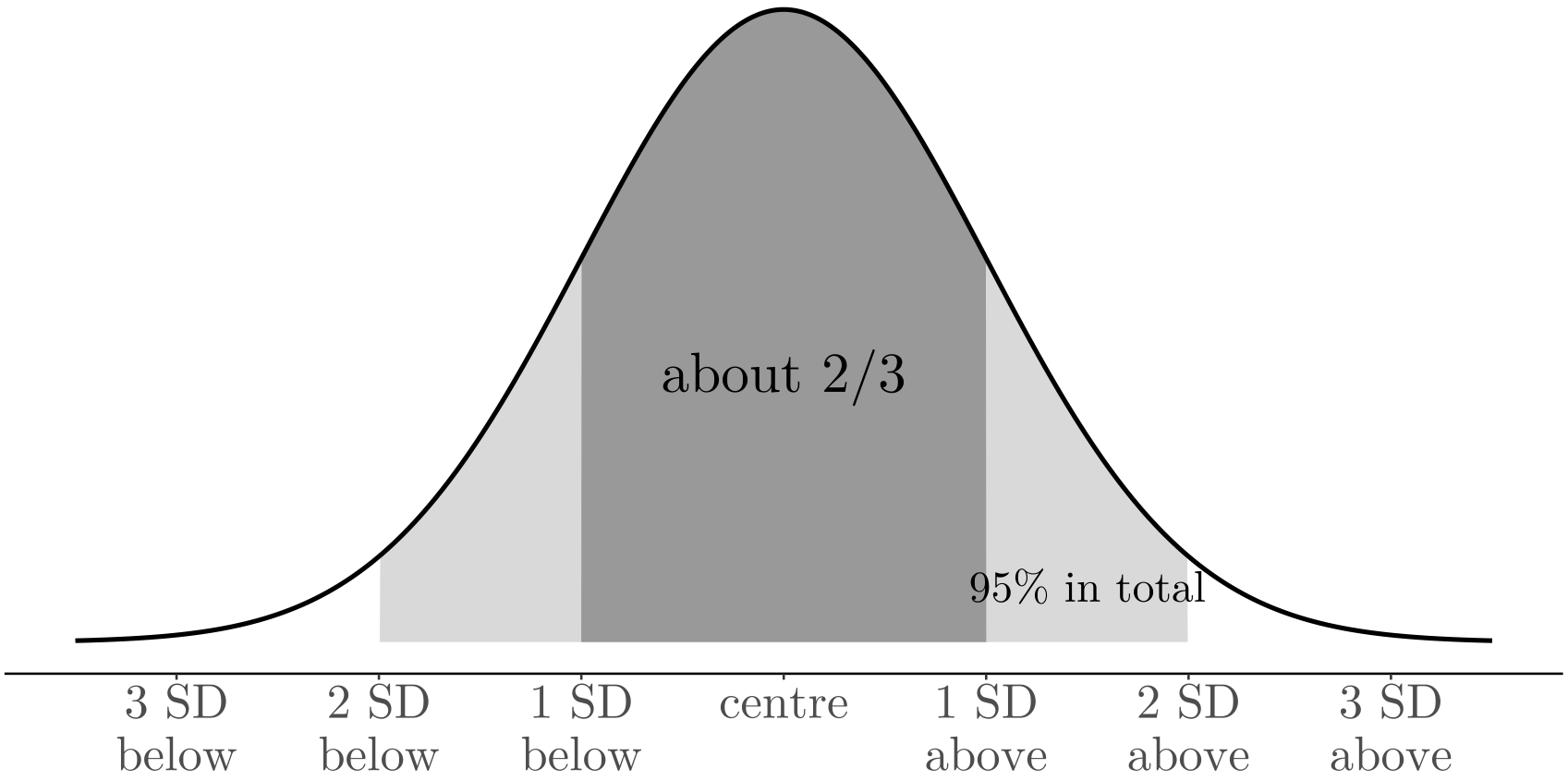
- Symmetric, with a single peak. Two numbers describe it completely: its **centre** and its **spread**.
- About **two thirds** of values fall within one standard deviation of the centre; about **95%** within two.
- Any bell curve can be rescaled into a standard one by asking: *how many standard deviations from the centre is this value?* That is all a "z-score" is.

It matters less because real variables follow a bell curve — most do not — and more because, as we will see in a moment, *averages from samples* do.

4.6 Centre and spread



4.7 Two thirds and 95 percent



4.8 Distributions in R: four prefixes

R names distribution functions systematically. For the bell curve (`norm`):

`dnorm(x)`	how high the curve is at `x`
`pnorm(q)`	the chance of a value at or below `q`
`qnorm(p)`	the reverse: the value below which a share `p` falls
`rnorm(n)`	`n` random draws

The same pattern holds for `binom`, `t`, `unif`, and the rest. You will see these in every script the agent writes: know what each one returns.

```
qnorm(0.975)                    # the famous 1.96
## [1] 1.959964
pnorm(1.96) - pnorm(-1.96)     # share within 1.96 SD
## [1] 0.9500042
```

Section 5

5. Lab: simulating distributions

5.1 Lab 2: specify, run, verify

To the agent

Write scripts/lab2.R. Draw 1,000 values from a bell curve centred on 50 with standard deviation 10; report their average, standard deviation and share within 10 of 50; plot a histogram. Then draw 20 values, three times, and report each average. Run it.

Verify — without asking the agent:

1. Open the script. Which function draws the data? Check its arguments against `?rnorm`: did 10 go in as the standard deviation, or as something else?
2. Compare the share within 10 of 50 to the two thirds we expect. Why is it not exactly two thirds?
3. Why do the three averages from 20 values differ so much more than the average from 1,000?
4. Now have the agent simulate how often 60 or more of 216 states would sign if each signs with a 28% chance. Check its answer against the exact value: `1 - pbinom(59, 216, 0.28)`.

Section 6

6. Why bigger samples help, and how much

6.1 Bigger samples settle down

As a sample grows, its average gets closer and closer to the true population average. (This is the **Law of Large Numbers**.)

- It licenses the whole enterprise: with enough data, estimates stop wandering.

6.1 Bigger samples settle down

As a sample grows, its average gets closer and closer to the true population average. (This is the **Law of Large Numbers**.)

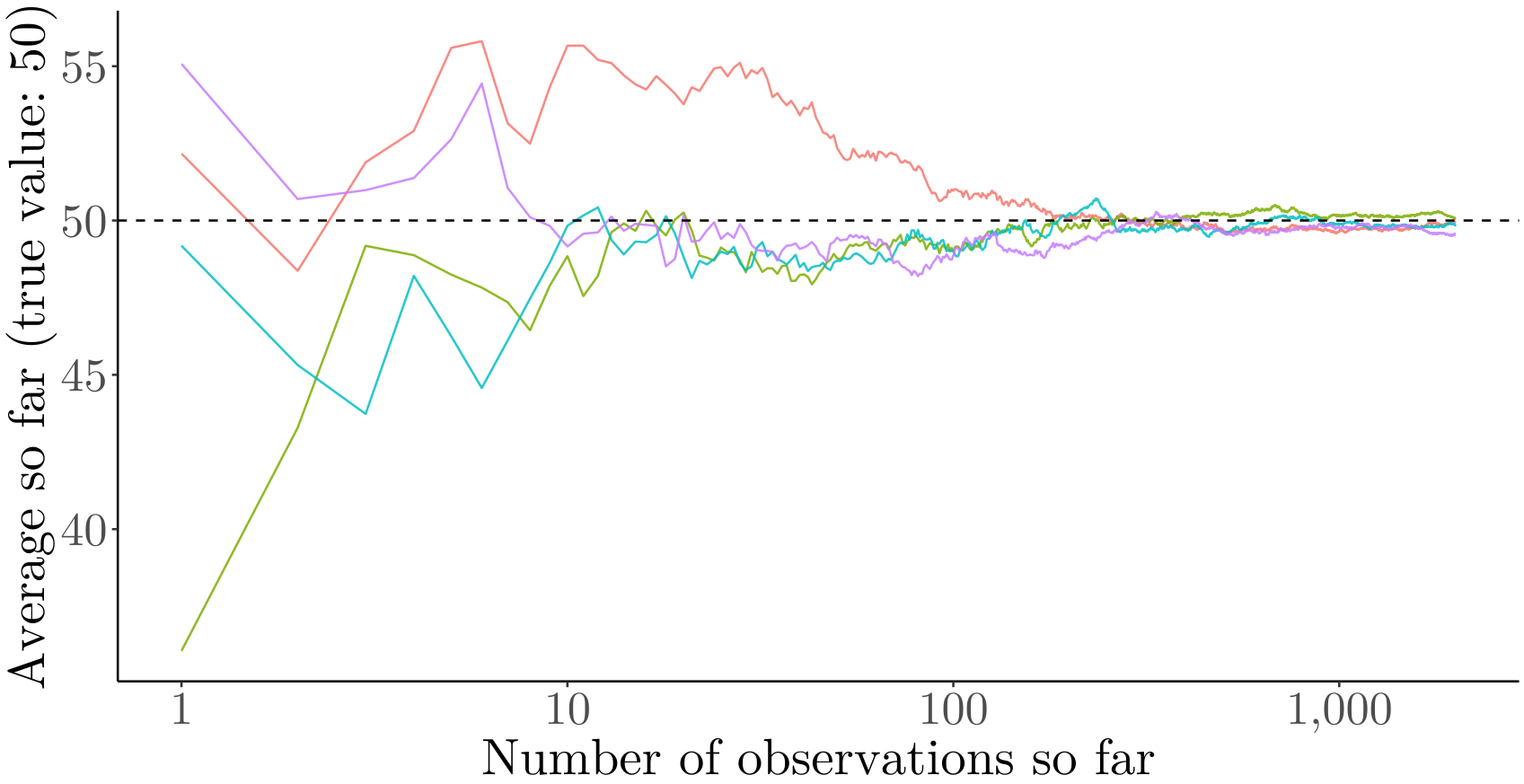
- It licenses the whole enterprise: with enough data, estimates stop wandering.
- It says nothing about *how fast*, and nothing about small samples.

6.1 Bigger samples settle down

As a sample grows, its average gets closer and closer to the true population average. (This is the **Law of Large Numbers**.)

- It licenses the whole enterprise: with enough data, estimates stop wandering.
- It says nothing about *how fast*, and nothing about small samples.
- It is a statement about the **method**, not about any single sample. A large sample can still be wrong; it is just less likely to be wrong by much.

6.2 Bigger samples settle down



6.3 Averages pile up in a bell curve

Knowing that averages settle down is not enough. We need to know *how* they scatter around the truth. (This is the **Central Limit Theorem**.)

- Imagine drawing many samples of the same size and computing the average of each. Those averages pile up in a **bell curve centred on the true value**.

6.3 Averages pile up in a bell curve

Knowing that averages settle down is not enough. We need to know *how* they scatter around the truth. (This is the **Central Limit Theorem**.)

- Imagine drawing many samples of the same size and computing the average of each. Those averages pile up in a **bell curve centred on the true value**.
- This happens **whatever the shape of the population**: lopsided, lumpy, yes/no — as long as samples are big enough.

6.3 Averages pile up in a bell curve

Knowing that averages settle down is not enough. We need to know *how* they scatter around the truth. (This is the **Central Limit Theorem**.)

- Imagine drawing many samples of the same size and computing the average of each. Those averages pile up in a **bell curve centred on the true value**.
- This happens **whatever the shape of the population**: lopsided, lumpy, yes/no — as long as samples are big enough.
- The spread of that pile is the **standard error**. It shrinks as samples grow: **four times the data halves the error**.

6.4 Why this is the most important slide in the module

If we know our average comes from a bell curve centred on the truth, with a spread we can estimate, then we can:

1. Say how far our estimate is *likely* to be from the truth — and turn that around into a range of plausible true values: a **confidence interval**.

That turn-around is the whole content of Session 2.

6.4 Why this is the most important slide in the module

If we know our average comes from a bell curve centred on the truth, with a spread we can estimate, then we can:

1. Say how far our estimate is *likely* to be from the truth — and turn that around into a range of plausible true values: a **confidence interval**.
2. Ask how surprising our estimate would be if the true effect were zero — a **hypothesis test**.

That turn-around is the whole content of Session 2.

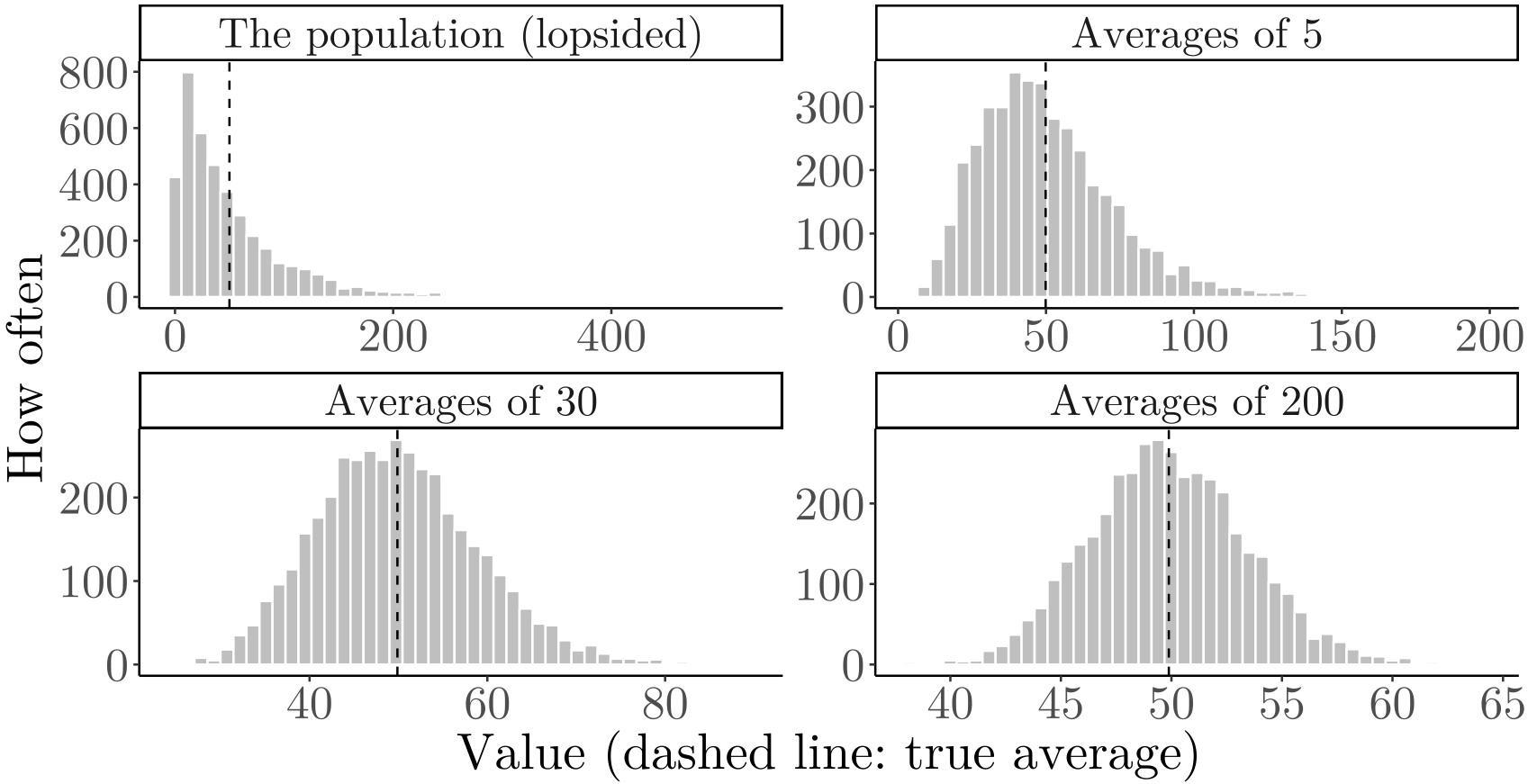
6.4 Why this is the most important slide in the module

If we know our average comes from a bell curve centred on the truth, with a spread we can estimate, then we can:

1. Say how far our estimate is *likely* to be from the truth — and turn that around into a range of plausible true values: a **confidence interval**.
2. Ask how surprising our estimate would be if the true effect were zero — a **hypothesis test**.
3. Do both without ever observing the population.

That turn-around is the whole content of Session 2.

6.5 Seeing it happen



6.6 A caution for your own work

"Big enough" depends on how badly behaved the population is.

- For a roughly symmetric population, about 30 observations is already fine.

6.6 A caution for your own work

"Big enough" depends on how badly behaved the population is.

- For a roughly symmetric population, about 30 observations is already fine.
- For something like `troops_iraq` in today's data — where almost every state contributes zero and one contributes 466,985 — 30 is nowhere near enough.

6.6 A caution for your own work

"Big enough" depends on how badly behaved the population is.

- For a roughly symmetric population, about 30 observations is already fine.
- For something like `troops_iraq` in today's data — where almost every state contributes zero and one contributes 466,985 — 30 is nowhere near enough.
- Small samples and extreme values are exactly where published findings tend to break. Be suspicious of a bell-curve approximation when samples are small and the outcome is lopsided.

Section 7

7. Lab: averages on real data

7.1 Real data

The course data for Modules I and II come from the replication archive for Goldsmith & Horiuchi (2012), *In Search of Soft Power* — the study from today's opening diagram. 225 states, one snapshot, 2002–03.

```
library(haven)
gh <- read_dta("data_2011-07-26.dta")
dim(gh)   # rows (states), columns (variables)
```

```
[1] 225  28
```

Variables we will use: `pos1` (share with a positive view of the US), `unvoting` (% agreement with the US in the UN General Assembly, 2003), `article98` (signed an Article 98 agreement), `icc` (ICC member), `nato`, `europe`, `lngdppc` (GDP per capita).

7.2 Lab 3: averages on real data

To the agent

Treat the observed values of unvoting in data_2011-07-26.dta as a population. Draw 2,000 random samples of 10 states, plot the averages of those samples, and report how spread out they are (standard deviation). Repeat with samples of 40. Run it.

Verify:

1. How many states have a value for `unvoting`? Did the agent drop the missing ones — and tell you?
2. Can the same state appear twice in one sample? Would it matter here?
3. Is there a `set.seed()`? Run it twice: same numbers?
4. Quadrupling the sample should halve the spread. Does it? Check the agent's numbers against `sd(gh$unvoting, na.rm = TRUE) / sqrt(10)` and `/ sqrt(40)`.
5. Add one line to `log.md`. Reference code: `Session-01-solutions.R` — only after verifying.

7.3 For next time

Read:

- Keele (2015), Sections 1–3 — the argument of today's opening (the decoder on the next slide translates its notation)
- Dean & Illowsky (2018), Chapters 3–7; Imai (2017), Chapter 6
- *Recommended:* Keele, Stevenson & Elwert (2020); Dowding & Miller (2026)

Do:

- Finish Lab 3 and add a line to `log.md`
- Bring a working installation, with OpenCode running

Session 2: we turn the bell curve of averages around. Its spread becomes a standard error, the standard error becomes a confidence interval, and the interval becomes a hypothesis test.

7.4 Decoder: notation you will meet in the readings

In the readings	**What it means on today's slides**
D_i	whether unit i got the treatment (1 = yes, 0 = no)
$Y_i(1)$, $Y_i(0)$	outcome in World 1 / World 2 (potential outcomes)
$E[\cdot]$	the average
ATE, ATT	the average effect, for all units / for the treated only
$E[Y \mid D = 1]$	the average outcome among the treated
$\perp$	"has nothing to do with" (independent of)
μ, π, β	true values: average, share, regression slope
$\bar{x}$, $\hat{p}$, $\hat{\beta}$	estimates of the same (a "hat" means estimated)
σ, s	standard deviation (true / estimated)
N, n	population size / sample size

You do not need to manipulate these. You need to recognise them, so that Keele (2015) reads like today's session.